BENCHMARK INSIGHTS REPORT

Why Agent Benchmarks Need Trajectory Intelligence

Evidence from Terminal-Bench, SWE-agent trajectories, and AFWB Foundation

Pass rate is useful, but incomplete. The benchmark evidence shows that agent teams also need trajectory, cost, waste, root-cause, and scaffold-level analysis.

Dataset family	Coverage used	Primary use in this report
Terminal-Bench	52,104 runs; cost and duration available	FinOps, failure cost, pass/fail, trajectory waste where available
SWE-agent	80,036 runs; full trajectories; no cost/duration	Edit/test loop behavior, repeat behavior, scaffold and trajectory divergence
AFWB Foundation	4 deterministic RCA scenarios	Controlled root-cause and remediation examples

Reading note

Dollar figures in this report come only from Terminal-Bench because that dataset has cost fields. SWE-agent is used for trajectory behavior only, and AFWB is used as deterministic RCA proof.

Executive Summary

Core finding

Agent quality cannot be evaluated by final pass/fail alone. Failed runs are expensive, successful runs can still be wasteful, and the same model can behave very differently under different agent scaffolds.

Metric	Value	Why it matters
Terminal-Bench pass rate	39.6%	A useful outcome metric, but not enough to explain cost or failure mode.
Terminal-Bench trajectory coverage	66.1%	About one-third of runs lack usable trajectories, which limits RCA.
SWE-agent pass rate	16.7%	The full processed SWE-agent data shows harder software-repair trajectories.
SWE-agent cost and duration	Unavailable	This prevents honest FinOps claims on SWE-agent alone.
Conservative early-stop savings	$1,495.83 and 452.8 hours	Counterfactual historical estimate from Terminal-Bench.

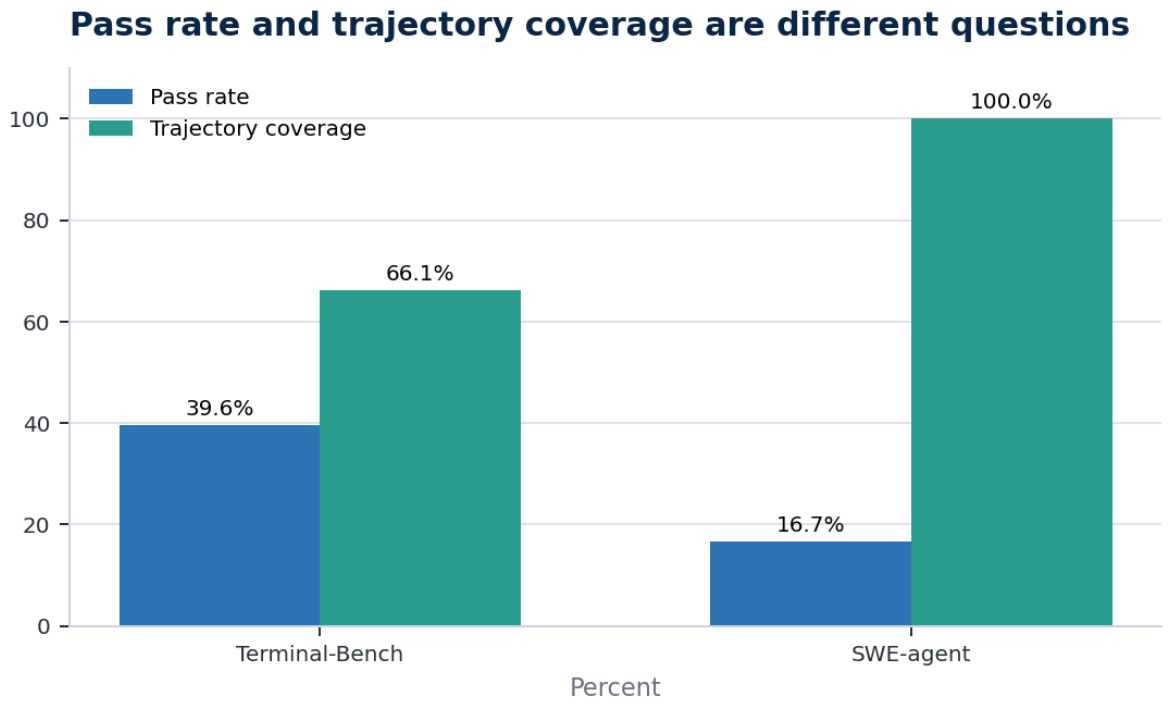

Figure 1. Pass rate and trajectory coverage answer different benchmark questions.

The report keeps dataset families separate. Terminal-Bench supports cost and duration analysis; SWE-agent supports trajectory-loop analysis; AFWB supports controlled RCA and remediation examples.

Methodology And Caveats

The analysis uses local processed artifacts from the benchmark repository. It does not merge dataset families into a universal leaderboard because each dataset has different coverage, metadata, and measurement semantics.

Dataset	Source revision / cutoff	What is measured	Important caveat
Terminal-Bench	04e8940f5b67...; cutoff 2026-03-05T22:54:39.383638+00:00	Outcome, trajectories when available, cost, duration, tokens	Only 66.1% of runs have usable trajectory steps.
SWE-agent	68195a145086...; source last modified 2024-12-23T12:42:05+00:00	Outcome and full normalized trajectories	Cost, duration, and token fields are unavailable in the processed source.
AFWB Foundation	Local deterministic foundation suite	Known root cause, first observable failure, remediation, avoidable usage	Controlled RCA proof layer, not public-scale benchmark evidence.

Counterfactual note

Early-stopping savings are historical counterfactual estimates: they estimate what could have been saved on these recorded trajectories if a stop rule had been applied.

1. What Is The Cost Of Failed AI Agent Runs?

Direct answer

Terminal-Bench failed runs consumed $17,845 in observed cost, about 82.8% of all observed run cost. Failed runs were 3.2x more expensive on average than successful runs: $0.567 vs. $0.180.

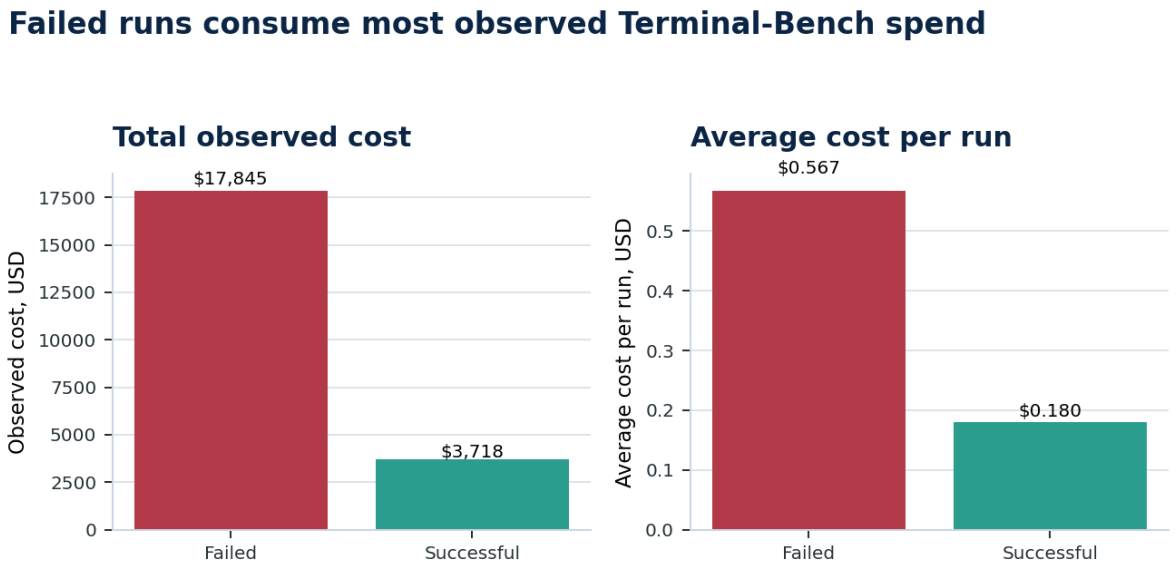

Figure 2. Failed runs dominate observed Terminal-Bench spend.

Metric	Value	Why it matters
Failed runs	31,455 runs; $17,845 observed cost	Failure burns budget before the final negative outcome.
Successful runs	20,649 runs; $3,718 observed cost	Successful outcomes still need efficiency analysis.
Median duration	565 sec failed vs. 307 sec successful	Failed runs tend to last longer.

Interpretation: cost-of-failure analysis should be a first-class benchmark dimension. A pass-rate-only table hides where the budget actually goes.

2. Why Can Successful Agent Runs Still Be Wasteful?

Direct answer

Successful Terminal-Bench trajectory runs still contained 5,107 suspected wasted actions, 27,070 error events, and 62,132 adjacent repeats. Estimated waste inside successful runs was $30.81 and 24.7 hours.

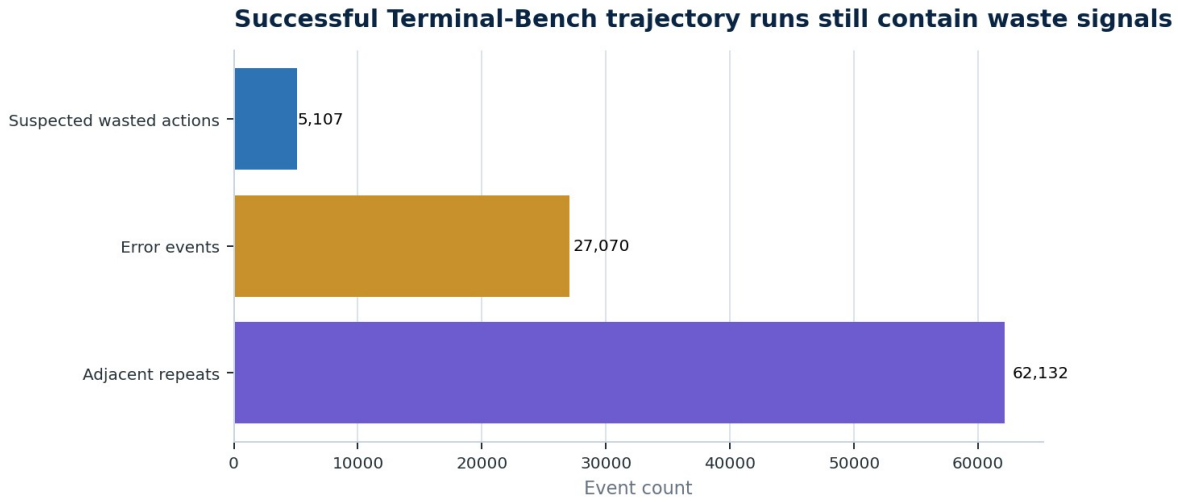

Figure 3. Passing runs can still contain repeated actions and error recovery overhead.

Interpretation: a passing run can still be brittle, slow, or expensive. The useful question is not only whether the agent eventually succeeded, but how much avoidable work it did on the way.

3. Which Coding Agents Repeat Failed Actions Most Often?

Direct answer

Strict repeated-failed-approach RCA is concentrated in a small number of agents. In rate terms, judy is highest at 9.55% of trajectory runs, but with a small sample. In count terms, mini-swe-agent has 55 runs and terminus-2 has 42 runs.

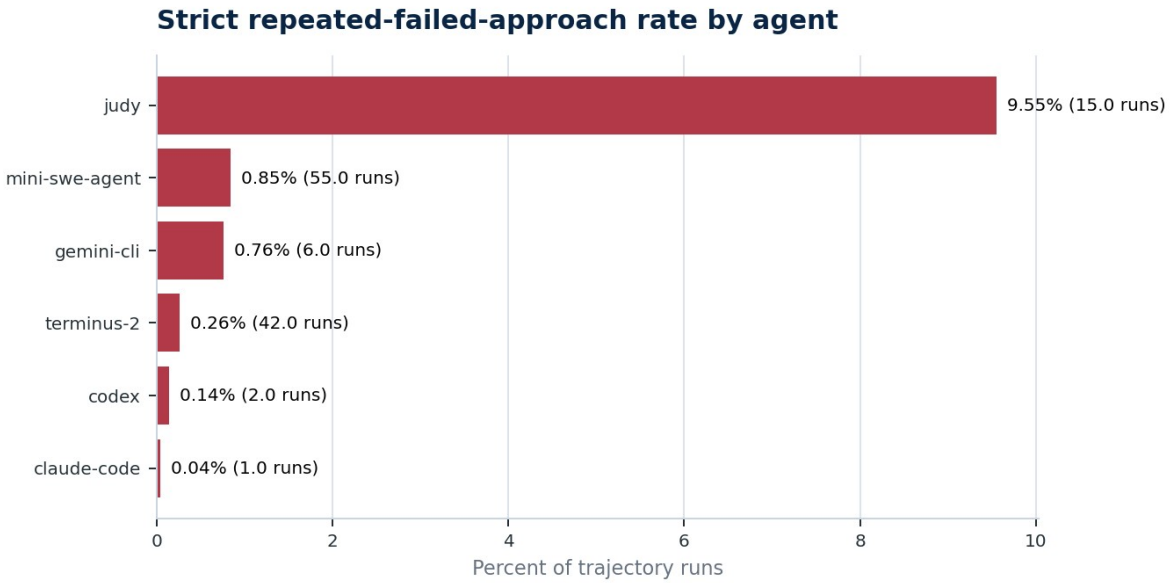

Figure 4. Strict repeated-failed-approach rate among agents with at least 100 trajectory runs.

Agent	Trajectory runs	Strict repeated-failed runs	Rate	Repeat behavior note
judy	157	15	9.6%	High strict RCA rate
mini-swe-agent	6,495	55	0.8%	High volume
gemini-cli	787	6	0.8%	Observed pattern
terminus-2	15,941	42	0.3%	High volume
codex	1,410	2	0.1%	Observed pattern
claude-code	2,522	1	0.0%	Observed pattern

For general repeat behavior, not only strict failed repeats, the highest mean repeat rates were judy at 54.5% and openhands at 47.7%.

Interpretation: repeated actions are invisible from final outcome alone. They require step-level trajectory analysis.

4. Where Do Successful And Failed Trajectories Diverge?

Direct answer

Terminal-Bench failed trajectory runs had higher medians than successful runs: 499 sec vs. 283 sec, 22 vs. 18 steps, and 25 vs. 23 tool calls. SWE-agent failed runs also used more steps and tool calls, with repeat rate 7.9% vs. 1.8%.

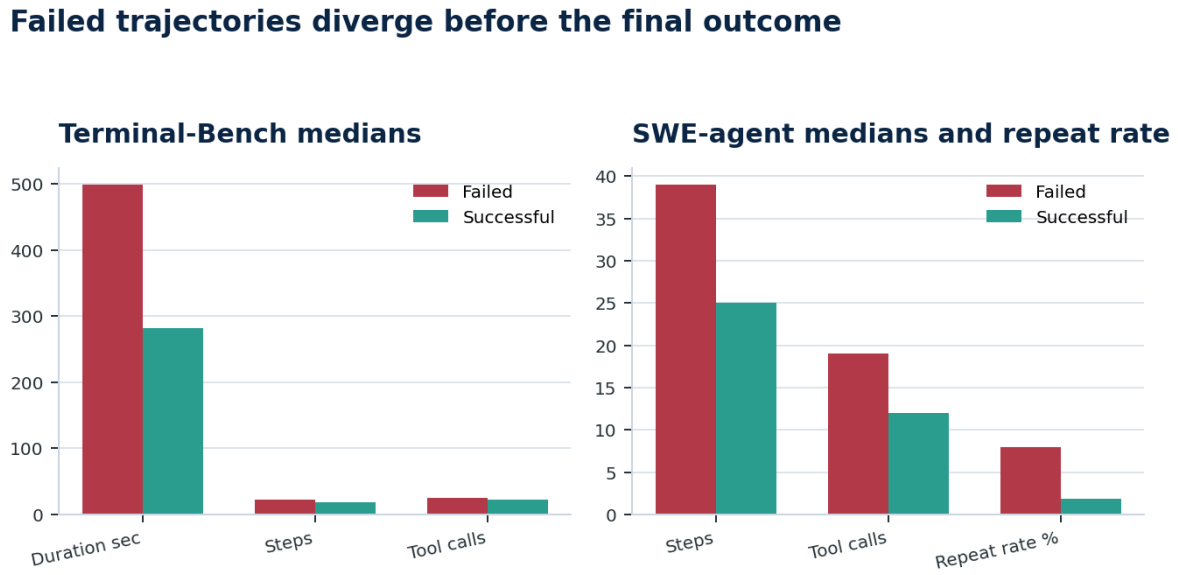

Figure 5. Failed trajectories show heavier execution patterns before final outcome.

Dataset	Failed trajectory profile	Successful trajectory profile	Most useful signal
Terminal-Bench	499 sec; 22 steps; 25 tool calls	283 sec; 18 steps; 23 tool calls	Error rate 10.0% failed vs. 7.6% successful
SWE-agent	39 steps; 19 tool calls	25 steps; 12 tool calls	Repeat rate 7.9% failed vs. 1.8% successful

Interpretation: trajectory signals can reveal a run going bad before the final answer is submitted.

5. How Much Could Early Stopping Save?

Direct answer

On Terminal-Bench, conservative early stopping could have saved about $1,495.83, 121,855 trajectory events, and 452.8 hours. It would trigger on 2,288 runs, or 4.4% of all runs.

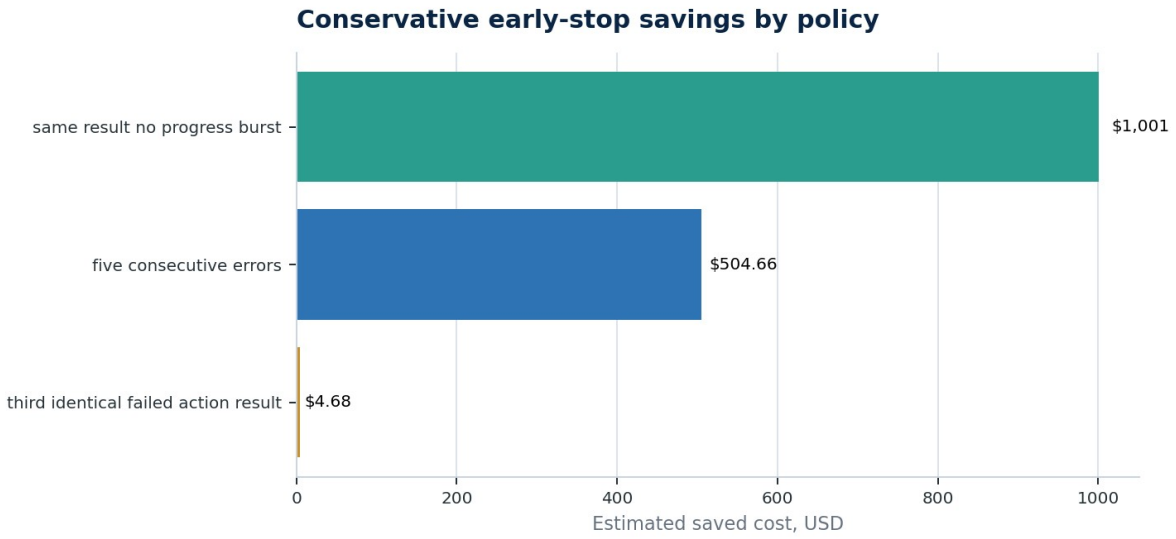

Figure 6. Conservative early-stop savings by individual policy.

Policy	Conservative triggers	Saved events	Estimated saved cost	Estimated saved duration
same result no progress burst	840 (1.6%)	52,002	$1,001.14	209.3 hours
five consecutive errors	1,516 (2.9%)	72,972	$504.66	256.7 hours
third identical failed action result	46 (0.1%)	1,863	$4.68	7.6 hours

Interpretation: stopping policy is a concrete efficiency lever. The best single policy in this run was same_result_no_progress_burst, saving about $1,001 conservatively.

6. Does The Same Model Behave Differently Under Different Scaffolds?

Direct answer

Yes. In Terminal-Bench trajectory runs for gpt-5@openai, pass rate ranged from 53.1% under codex to about 35.0% under mini-swe-agent, with large duration differences across scaffolds.

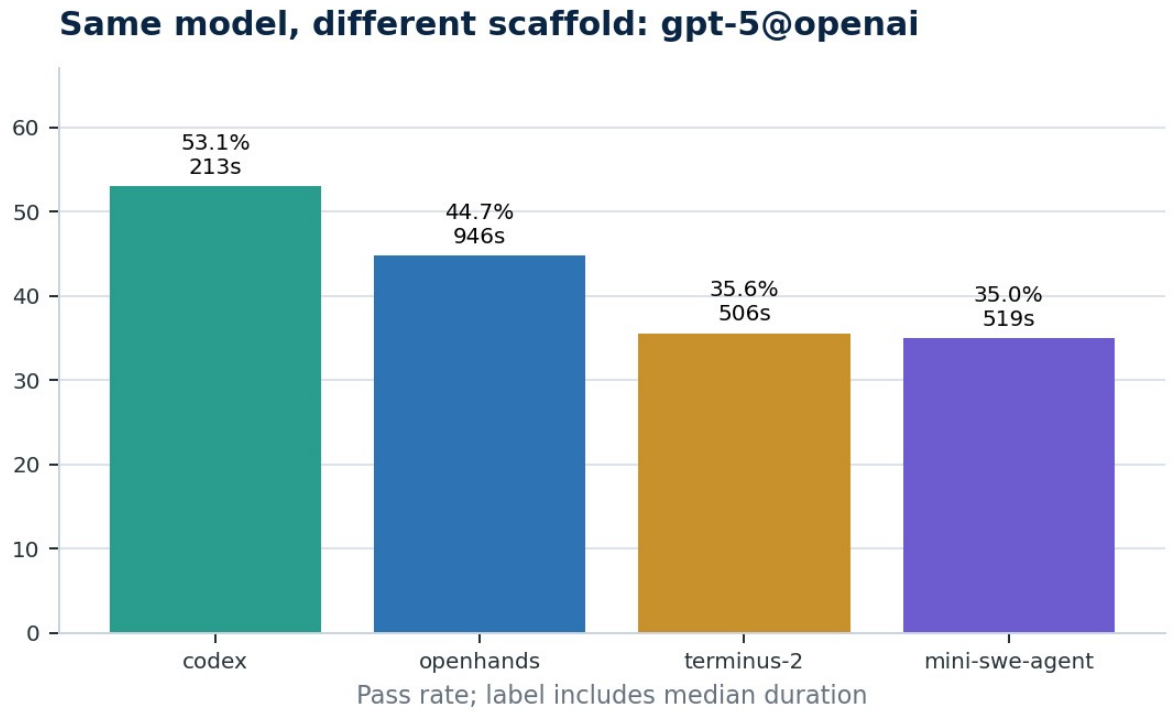

Figure 7. Same model, different scaffold outcomes for gpt-5@openai.

Agent scaffold	Runs	Pass rate	Median duration	Median steps	Median tool calls
codex	409	53.1%	213 sec	17	12
openhands	436	44.7%	946 sec	41	30
terminus-2	436	35.6%	506 sec	7	25
mini-swe-agent	432	35.0%	519 sec	9	7

Interpretation: model-only benchmarking is incomplete. The harness, tools, context strategy, retry behavior, and verification loop can materially change the outcome for the same model.

7. What Are The Most Expensive Failure Patterns?

Direct answer

By estimated avoidable waste, possible missing verification and tool execution failure are the top Terminal-Bench patterns. By total observed spend trapped in failed runs, tool execution failure is much larger, with about $9,522 in observed cost.

Figure 8. Estimated avoidable Terminal-Bench cost by failure pattern.

Failure pattern	Runs	Failed runs	Estimated avoidable cost	Observed cost in pattern
Possible Missing Verification	7,630	7,630	$75.76	$3,541
Tool Execution Failure	9,034	9,034	$65.53	$9,522
No Major Rule Based Failure	11,079	0	$30.81	$3,435
Excessive Exploration	557	557	$22.54	$939
Unclassified Failure	5,774	5,774	$11.21	$1,406
Repeated Failed Approach	121	121	$9.68	$70
Late Failure	267	267	$1.30	$2,082

Interpretation: teams should prioritize failure patterns by economic impact, not only by count. Tool execution failure has 9,034 runs and $9,522 observed cost in this analysis.

8. Why Is Pass Rate Alone A Weak Agent Benchmark?

Direct answer

Terminal-Bench pass rate is 39.6%, but only 66.1% of runs have usable trajectories. SWE-agent pass rate is 16.7% on 80,036 full processed runs, but cost and duration are unavailable. Pass rate answers whether a run worked; it does not answer why, at what cost, or what system behavior caused the result.

Metric	Value	Why it matters
Outcome	Pass/fail	Useful but too coarse for system improvement.
Observability	Terminal-Bench trajectory coverage 66.1%; SWE-agent 100.0%	A run without a trajectory cannot support detailed RCA.
FinOps	Terminal-Bench cost available; SWE-agent cost unavailable	Cost-per-success claims need measured cost fields.
Root cause	Tool errors, missing verification, repeats, late failures	These require step-level signals, not only final reward.
Cutoff and metadata	SWE-agent source last modified 2024-12-23T12:42:05+00:00; snapshot cutoff unavailable	Benchmark claims should disclose coverage and missing metadata.

Interpretation: stronger agent evaluation combines pass rate with trajectory coverage, cost per success, wasted actions, error events, repeat behavior, verification behavior, and root-cause classification.

Controlled RCA Proof: AFWB Foundation

Why this matters

AFWB Foundation is not a public-scale benchmark. It is a deterministic proof layer with 4 scenarios that name the root cause, first observable failure, remediation, and avoidable usage.

Scenario	Root cause	Remediation	Avoidable usage
invalid_booking_date	TOOL_ARGUMENT_ERROR	ARGUMENT_SCHEMA_VALIDATION	1 tool calls; 125 ms; 114 tokens
duplicate_payment	SIDE_EFFECT_DUPLICATION	IDEMPOTENCY_KEY	1 tool calls; 214 ms; 100 tokens
infinite_retry_terminal_error	TOOL_RETRY_STORM	RETRY_POLICY_CHANGE	2 tool calls; 190 ms; 0 tokens
missing_loop_exit_condition	LOOP_NON_TERMINATION	TERMINATION_CONDITION	1 tool calls; 152 ms; 128 tokens

Metric	Value	Why it matters
Avoidable tool calls	5	Shows excess tool usage in controlled failures.
Avoidable latency	681 ms	Measures delay from preventable behavior.
Avoidable tokens	342	Captures prompt/output overhead in controlled failures.

What These Benchmarks Let Teams Ask

The common thread across the datasets is not a single universal ranking. It is a set of operational questions that pass rate alone cannot answer.

Operational question	Benchmark evidence	Why the answer matters
Where is budget being wasted?	Terminal-Bench cost, duration, and waste estimates	Prioritize failures by dollars and time.
Which failures are preventable?	AFWB root-cause and remediation labels	Separate model limits from system design issues.
Which runs should stop earlier?	Terminal-Bench early-stop simulations	Reduce cost and delay from hopeless loops.
Which successful runs are fragile?	Successful-run errors, repeats, and wasted actions	Improve reliability even when pass rate looks acceptable.
Is the model or scaffold responsible?	Same-model, different-scaffold comparison	Tune harness and tool interface before defaulting to model replacement.
Can we trust the benchmark row?	Trajectory, cost, duration, and cutoff coverage	Avoid making claims from missing or incompatible fields.

Do not combine these into one universal leaderboard

Terminal-Bench, SWE-agent, and AFWB answer different questions. A credible evaluation keeps benchmark families separate and compares systems only within compatible measurement contexts.

How Lumniverse Helps

Solution thesis

Lumniverse turns agent trajectories into operational intelligence: why agents fail, where they waste money and time, which scaffold behaviors drive outcomes, and what teams should change next.

Problem exposed by the benchmarks	Lumniverse capability	Practical outcome
Failed runs burn budget before failing	Cost and waste attribution by run, task, model, and scaffold	Teams can quantify the economic impact of failure.
Successful runs hide waste	Trajectory-level efficiency scoring	Teams can optimize passing runs, not only failing ones.
Agents repeat failed actions	Loop, retry, and same-result detection	Teams can set better stop, retry, and escalation policies.
Model-only benchmarks mislead	Model + scaffold + tool comparison	Teams can improve harness design instead of only swapping models.
Root cause is unclear	RCA for tool, context, verification, retry, and submit failures	Teams get actionable explanations, not just aggregate scores.
Benchmark findings do not automatically transfer to production	Benchmark-to-production observability layer	Teams can monitor the same failure modes in live agent systems.

The product value is deliberately downstream of the evidence: benchmarks reveal that pass rate is insufficient; Lumniverse gives teams the trajectory intelligence needed to act on that reality.